DSA CASE STUDY

GRAPH-BASED FRAUD DETECTION IN FINANCIAL TRANSACTION NETWORKS

Application of Graph Traversal and Network Analysis Algorithms

A CASE STUDY ON DATA STRUCTURES AND ALGORITHMS (DSA)

Submitted in partial fulfilment of the requirements for the subject Data Structures and Algorithms (DSA)

NAME
Manyu Kumar

REGISTRATION NUMBER
2502051966

PROGRAMME
B.Tech CSE

SECTION
N3

1. ABSTRACT

Financial fraud has become a complex challenge as digital banking, online payments, cards, and electronic fund transfers connect large numbers of accounts and other financial entities. Conventional systems often examine transactions using individual attributes such as amount, time, location, or predefined rules. Although useful, this approach may miss patterns that emerge only when several related transactions are considered together.

This case study examines how Data Structures and Algorithms (DSA), particularly graph-based techniques, can support the analysis of financial transaction networks. In the model, accounts and other entities are represented as vertices, while transactions or relationships are represented as edges. Directed edges preserve the direction of money movement, and edge attributes may record transaction amount, timestamp, frequency, or a calculated cost.

The report covers graph fundamentals, graph representations, Breadth-First Search (BFS), Depth-First Search (DFS), cycle detection, connected components, strongly connected components, shortest-path algorithms, centrality measures, and community detection. Supporting structures—including hash tables, sets, queues, stacks, heaps, and priority queues—are considered alongside time and space complexity.

A hypothetical transaction network is used to demonstrate how graph algorithms can reveal transaction chains, circular relationships, shared intermediaries, and groups that may warrant investigation. The report also compares graph-based analysis with transaction-level approaches and discusses data quality, scalability, privacy, interpretability, and false positives.

The central conclusion is that graph analysis provides a valuable relationship-aware complement to conventional fraud detection. A graph pattern is not proof of wrongdoing; it is a signal that should be evaluated alongside transaction attributes, context, and appropriate human investigation.

TABLE OF CONTENTS

1. ABSTRACT 2
2. INTRODUCTION 3
3. FINANCIAL FRAUD AND PROBLEM DOMAIN 4
4. GRAPH THEORY AND DSA FOUNDATION 5
5. REPRESENTING A FINANCIAL SYSTEM AS A GRAPH 9
6. GRAPH-BASED FRAUD DETECTION METHODOLOGY 10
7. FRAUD PATTERNS IN TRANSACTION GRAPHS 11
8. ILLUSTRATIVE CASE STUDY AND TRANSACTION NETWORK 12
9. ALGORITHMIC AND COMPLEXITY ANALYSIS 15
10. COMPARATIVE ANALYSIS 16
11. LIMITATIONS, RISKS, AND CHALLENGES 17
12. ADVANCED EXTENSIONS AND FUTURE SCOPE 17
13. CONCLUSION 18
14. REFERENCES 19

2. INTRODUCTION

2.1 Background

Digital financial services allow customers and businesses to transfer money, make purchases, pay bills, receive salaries, and manage accounts electronically. These services create large volumes of records involving customers, bank accounts, cards, merchants, devices, and locations. The same connectivity that makes financial services convenient can also make suspicious activity difficult to understand when it is distributed across many transactions.

Financial fraud may involve one unauthorized payment, but coordinated activity can involve several accounts and intermediate transfers. Funds may pass through a sequence of accounts, or apparently separate accounts may share devices, contact details, or transaction partners. A single transaction may appear ordinary even when its position in a larger network is unusual.

2.2 Why Graphs Are Relevant

A graph is a natural representation for a system made up of entities and relationships. In a financial transaction graph, vertices represent accounts or other entities, and directed edges represent transactions. Attributes attached to edges can preserve amounts, timestamps, and transaction types. The graph can therefore show not only what happened but also how entities are connected.

Graph algorithms can explore nearby accounts, follow transaction chains, detect directed cycles, identify groups, and measure the structural position of entities. These operations complement transaction-level rules by adding network context.

2.3 Objectives

- Represent financial entities and transactions using appropriate graph structures.
- Explain BFS, DFS, cycle detection, connectivity, shortest paths, centrality, and communities.
- Demonstrate the algorithms using a hypothetical transaction network.
- Analyse algorithmic time and space requirements and compare graph-based and transaction-level approaches.
- Discuss practical limitations and responsible use of graph-derived alerts.

2.4 Scope

The report concentrates on DSA and network analysis. Machine learning and graph neural networks are discussed only as extensions. The example data are hypothetical, and the report does not claim to implement or validate a production banking system.

3. FINANCIAL FRAUD AND PROBLEM DOMAIN

3.1 Understanding Financial Fraud

Financial fraud involves intentional deception, unauthorized activity, or manipulation to obtain a financial benefit or cause loss. Detection is difficult because legitimate and fraudulent transactions can share similar attributes. A large payment may be valid, and a high transaction count may be normal for a merchant. Suspicion must therefore be based on context rather than one feature alone.

3.2 Common Types

- **Transaction and payment fraud:** unauthorized transfers or payments made using compromised credentials.
- **Account takeover:** an unauthorized person gains control of an account and performs activity.
- **Card fraud:** payment-card details are used without valid authorization.
- **Money laundering:** funds are moved through transactions or activities to conceal illicit origins.
- **Mule-account activity:** accounts receive or move funds for another party, sometimes without full awareness by the account holder.
- **Identity-related fraud:** personal information is misused to obtain services or conduct transactions.

Each category can produce relationships among accounts, devices, merchants, and transactions. A multi-entity graph can represent these links, but the presence of a relationship is not itself evidence of misconduct.

3.3 Limits of Conventional Detection

Rule-based systems apply predefined conditions, such as an amount threshold or a high transaction frequency. Transaction-level models evaluate each record independently. These methods are useful for known patterns but can miss coordinated activity spread across multiple accounts. Static thresholds may also create false positives when they ignore normal differences between customer types.

A false positive occurs when legitimate activity is flagged as suspicious; a false negative occurs when fraudulent activity is missed. Graph analysis can provide additional evidence, but it does not eliminate either problem. Results still require appropriate thresholds, contextual information, and review.

3.4 Need for Network-Level Analysis

Network-level analysis asks how a transaction relates to other transactions and entities. It can examine chains, cycles, shared intermediaries, connected groups, and changes in account relationships. The objective is not to label every unusual structure as fraud, but to find patterns that merit a closer look.

4. GRAPH THEORY AND DSA FOUNDATION

4.1 Definition and Notation

A graph is commonly written as G = (V, E), where V is the set of vertices and E is the set of edges. Vertices represent entities; edges represent relationships. In a transaction graph, an edge A → B means a transaction from account A to account B.

4.2 Key Terms

- **Vertex:** an entity represented in the graph.
- **Edge:** a relationship between two vertices.
- **Degree:** number of incident edges in an undirected graph.
- **In-degree:** number of directed edges entering a vertex.
- **Out-degree:** number of directed edges leaving a vertex.
- **Path:** a sequence of vertices connected by edges.
- **Walk:** a sequence in which vertices or edges may repeat.
- **Trail:** a walk that does not repeat an edge.
- **Cycle:** a closed path that returns to its starting vertex.
- **Subgraph:** a graph formed from a subset of vertices and edges.
- **Connectivity:** reachability among vertices under the graph's direction rules.

These definitions allow an analyst to describe a network precisely. For example, a high out-degree account sends transactions to many destinations, while a cycle shows that a directed sequence returns to its origin. Neither property independently proves fraud.

4.3 Graph Types

Directed graphs preserve the direction of transactions. Undirected graphs are suitable for relationships where direction is not meaningful, such as a shared device. Weighted graphs attach values to edges, such as amount or a defined cost. Unweighted graphs treat each edge as one step. A multigraph can preserve multiple transactions between the same accounts, while a simple graph normally excludes parallel edges and self-loops. A dynamic graph changes as transactions or relationships are added, removed, or updated.

4.4 Graph Representations

4.4.1 Adjacency Matrix

An adjacency matrix is a V × V array. For a directed graph, entry M[i][j] records whether an edge exists from vertex i to vertex j; a weighted matrix can store the edge weight. The representation makes edge-existence checks O(1), but it requires $O(V^2)$ space and takes O(V) time to inspect all possible neighbours of a vertex.

4.4.2 Adjacency List

An adjacency list stores the outgoing neighbours of each vertex. It uses O(V + E) space and is efficient for exploring the neighbours of a vertex. With adjacency lists, BFS and DFS run in O(V + E) time over a complete traversal. This is often suitable for sparse transaction networks, where each account connects to only a small fraction of all accounts.

4.4.3 Edge List

An edge list stores each edge as a source, destination, and optional attributes. It is convenient for importing and processing transaction records sequentially, but finding all neighbours of a vertex may require scanning the edge list unless an index is maintained.

Representation	Space	Typical strength	Limitation
Adjacency matrix	$O(V^2)$	Fast edge check	High memory use on sparse graphs
Adjacency list	$O(V + E)$	Efficient traversal	Edge check depends on degree
Edge list	$O(E)$	Sequential processing	Neighbour search may be slow

Table 1: Comparison of graph representations.

For this case study, an adjacency list is the primary representation. The raw transaction records can still be stored as an edge list or table, then transformed into an adjacency structure for traversal.

4.5 Breadth-First Search (BFS)

Breadth-First Search explores a graph level by level from a source vertex. It uses a queue, which follows the First In, First Out principle. The source is marked visited and enqueued. The algorithm repeatedly removes the front vertex, examines its outgoing neighbours, and enqueues each unvisited neighbour.

```
BFS(G, source):
    mark all vertices unvisited
    create empty queue Q
    mark source visited; enqueue source
    while Q is not empty:
        u = dequeue Q
        for each neighbour v of u:
            if v is unvisited:
                mark v visited
                parent[v] = u
                distance[v] = distance[u] + 1
                enqueue v
```

For edges A→B, A→C, B→D, and C→E, a BFS starting at A may visit A, B, C, D, E, assuming B is processed before C. BFS finds minimum edge-count distances in an unweighted graph. It can identify accounts within one, two, or more transaction hops of a source. With an adjacency list, time complexity is $O(V + E)$ and auxiliary space is $O(V)$. The actual work for a partial search depends on the reachable portion of the graph.

4.6 Depth-First Search (DFS)

Depth-First Search explores one branch as deeply as possible before backtracking. It can be implemented recursively or with an explicit stack. For the same graph, a DFS starting at A might visit A, B, D, C, E. The exact order depends on neighbour ordering.

```
DFS(u):
    mark u visited
    for each neighbour v of u:
        if v is unvisited:
            parent[v] = u
            DFS(v)
```

DFS is useful for exploring long transaction chains, checking reachability, finding connected components, and supporting cycle and strongly connected component algorithms. Its time complexity with adjacency lists is $O(V + E)$, and auxiliary space is $O(V)$ for visited information and the recursion or explicit stack.

Deep recursive graphs can exceed the language's call-stack capacity, so iterative DFS may be preferred in some systems.

4.7 Cycle Detection

A directed cycle follows edge directions and returns to its starting vertex, for example A→B→C→A. DFS-based cycle detection can use three states: unvisited, visiting (on the active DFS path), and visited (fully processed). If an edge reaches a vertex in the visiting state, a back edge has been found and a directed cycle exists.

Cycle detection can identify circular transaction structures for review. However, legitimate payment, refund, and settlement activity can also produce cycles. Interpretation requires amounts, timestamps, account context, and additional evidence. With an adjacency list and three-state DFS, cycle detection takes O(V + E) time and O(V) auxiliary space.

4.8 Connected Components

In an undirected graph, a connected component is a maximal set of vertices where each vertex can reach every other vertex. Repeated BFS or DFS can find all components: start a traversal at each unvisited vertex, and assign every reached vertex to the same component. The total time is O(V + E).

For directed graphs, weak connectivity ignores edge direction when forming groups; strong connectivity respects direction. A strongly connected component (SCC) is a maximal group in which every vertex can reach every other vertex by following directed edges.

SCC algorithms

Kosaraju's algorithm performs a DFS to obtain finishing order, reverses all edges, and performs a second DFS in the required order. Tarjan's algorithm uses DFS discovery indices, low-link values, and a stack. Both run in O(V + E) time with adjacency lists and use O(V) auxiliary space.

SCCs may reveal mutual reachability or circular relationships, but a component is not automatically a fraudulent group. Graph direction, transaction time, and domain context must be preserved when interpreting the result.

4.9 Shortest-Path Algorithms

A shortest path minimizes a chosen cost. The cost may be the number of edges, total distance, time, or another defined value. The edge-weight meaning must be chosen carefully: if the goal is the fewest transaction steps, an unweighted graph and BFS are appropriate; if weights represent non-negative costs, Dijkstra's algorithm may be suitable.

- **BFS:** shortest paths by number of edges in an unweighted graph; O(V + E).
- **Dijkstra:** shortest paths with non-negative edge weights; O((V + E) log V) with an adjacency list and binary heap in a common implementation.
- **Bellman–Ford:** supports negative edge weights and can detect a reachable negative-weight cycle; O(VE).

A path that minimizes monetary amount is not necessarily the path that best explains suspicious behaviour. Shortest-path analysis must answer a clearly defined question.

4.10 Supporting Data Structures

- **Hash tables** map account identifiers to graph indices or metadata. Lookup and insertion are average-case O(1), though worst-case operations can be O(n).
- **Sets** store visited vertices and help prevent repeated exploration. Hash-based set operations are typically average-case O(1).
- **Queues** support BFS; **stacks** support iterative DFS and some SCC procedures.
- **Priority queues and binary heaps** process high-priority records efficiently. Heap insertion and removal typically take O(log n), while inspecting the top element takes O(1).
- **Sorting** ranks accounts or transactions by selected values. Efficient comparison sorts such as merge sort have O(n log n) time complexity.

These structures do not themselves determine whether fraud occurred. They help store, traverse, order, and prioritize the information produced by the detection workflow.

4.11 Network Analysis Concepts

Degree centrality

Degree centrality counts an entity’s direct connections. In a directed transaction graph, in-degree and out-degree should be considered separately. An account receiving from many accounts or sending to many accounts may deserve attention, but merchants and payment processors can naturally have high degrees.

Betweenness centrality

Betweenness centrality measures how often a vertex lies on shortest paths between pairs of other vertices. A high score may indicate an intermediary connecting different parts of a network. Exact computation can be expensive on very large graphs, so approximations or restricted subgraphs may be used.

Closeness centrality

Closeness centrality reflects the shortest-path distance from a vertex to other reachable vertices. Disconnected graphs require careful handling because some vertices cannot be reached. A high score describes structural proximity, not guilt.

Eigenvector centrality

Eigenvector centrality assigns greater importance to vertices connected to other important vertices. Its meaning depends on the network and calculation method, and it should not be interpreted as a direct measure of fraud.

Community detection

Community detection identifies groups with comparatively dense internal connections under a chosen method. Communities may correspond to legitimate businesses or customer networks as well as groups that warrant review. The algorithm, graph direction, and edge weighting influence the result.

Summary of the DSA foundation

Graphs model relationships; adjacency lists store sparse networks efficiently; BFS and DFS explore reachability; cycle and component algorithms identify structural patterns; shortest-path algorithms measure relationships under a defined cost; centrality and communities describe network organization. Complexity analysis helps determine whether the chosen methods are suitable for the graph’s size.

5. REPRESENTING A FINANCIAL SYSTEM AS A GRAPH

5.1 Entities as Vertices

The simplest model represents each bank account as a vertex. More detailed models can include customers, cards, merchants, devices, telephone numbers, addresses, and transactions. If different entity types are included, the graph becomes a multi-entity or heterogeneous graph. Each vertex should have a stable internal identifier and a type label.

5.2 Transactions as Directed Edges

A transfer from account A to account B is represented by A→B. The edge can carry attributes such as transaction ID, amount, timestamp, currency, channel, status, and transaction type. Direction is essential because receiving funds and sending funds have different meanings.

5.3 Example Edge Table

Transaction	From	To	Amount	Time
T1	A	B	₹9,00,000	09:00
T2	B	C	₹8,80,000	09:12
T3	C	D	₹8,70,000	09:25
T4	D	B	₹1,00,000	10:10

Table 2: Illustrative edge table.

This table is illustrative only. It demonstrates how transaction records can be converted into directed edges. It does not describe real accounts or a confirmed fraud incident.

5.4 Static and Temporal Views

A static graph represents a selected snapshot or period. A temporal graph preserves the order and timing of events. A rolling time window—for example, the previous hour or day—can focus analysis on recent activity. If timestamps are ignored, a graph may show a cycle even when the transactions occurred in an order that could not represent a single continuous flow of funds.

5.5 Data Preparation

- Validate transaction records and normalize timestamps and currencies.
- Remove duplicate records only when duplication is confirmed; retain legitimate repeated transactions.
- Resolve entity identifiers carefully to avoid merging different customers.
- Handle missing values and inconsistent identifiers transparently.
- Preserve provenance so an alert can be traced to the underlying records.

The quality of graph analysis depends on the quality of the data. Incorrect entity resolution can create false connections, while missing transactions can hide genuine relationships.

6. GRAPH-BASED FRAUD DETECTION METHODOLOGY

6.1 Proposed Workflow

The proposed workflow begins with transaction records and ends with a prioritized set of cases for review. It is a conceptual architecture for this case study rather than a deployed banking product.

- **Input:** transaction records and relevant entity information.
- **Preprocessing:** validate records, standardize fields, and resolve identifiers.
- **Graph construction:** create vertices, directed edges, and attributes.
- **Graph analysis:** run traversal, cycle, connectivity, path, and network measures.
- **Risk evaluation:** combine structural indicators with transaction context and established rules.
- **Alert generation:** store evidence and rank cases for human review.

6.2 Graph Construction

Each account is assigned a vertex. Every valid transfer creates a directed edge from the sending account to the receiving account. The edge stores the transaction amount and timestamp. Multiple transactions between the same accounts may be retained as parallel edges or aggregated, depending on the analysis. For time-sensitive analysis, transaction order must be preserved.

6.3 Traversal-Based Analysis

BFS can explore accounts within a specified number of transaction hops from a source. DFS can follow deeper paths and support cycle detection. A maximum depth or time window can limit work and keep the analysis relevant. The resulting paths should be recorded so an investigator can understand why an alert was raised.

6.4 Structural Indicators

Candidate indicators include unusual transaction chains, directed cycles, accounts with unusual in-degree or out-degree, groups with dense interactions, and sudden changes in connections. These indicators are features for investigation, not final labels.

6.5 Risk Prioritization

A system may combine indicators into a transparent risk score, provided the formula is documented and tested. A priority queue can then place high-scoring cases first. The queue only orders cases; it does not validate the score or prove that the underlying activity is fraudulent.

6.6 Interpretation and Alert Handling

A useful detection workflow should explain why a case was selected. An alert may include the source account, the relevant path or cycle, timestamps, amounts, the graph algorithm used, and the rules or indicators that contributed to its priority. Traceability helps reviewers distinguish a genuine concern from a data-quality issue.

Indicator	Possible meaning	Required caution
High out-degree	Many outgoing transaction links	May be normal for a merchant or business
Short, rapid chain	Funds move through several accounts quickly	Timing and purpose need verification
Directed cycle	A path returns to its origin	Legitimate refunds or settlements can form cycles
Shared device	Several accounts use one device	Family or business use may be legitimate
Dense community	Accounts interact frequently	Dense groups can be ordinary business networks

Table 3: Illustrative indicators.

6.7 Human Review

The system should support, not replace, a qualified investigation. Reviewers may check account history, customer context, transaction authorization, documentation, and other evidence. The final decision should not rely solely on graph structure or a single numerical score.

6.8 Privacy and Governance

Only data needed for a defined purpose should be processed. Access should be limited, sensitive identifiers should be protected, and retention should follow applicable policy and law. Entity matching must be auditable because an incorrect match can create misleading network connections.

6.9 Evaluation

A proposed system should be evaluated using appropriate historical cases, carefully labelled examples, and measures such as precision, recall, false-positive rate, processing time, and analyst workload. Evaluation data should be separated from the data used to tune rules to reduce overly optimistic results. No performance claim is made in this conceptual case study.

7. FRAUD PATTERNS IN TRANSACTION GRAPHS

7.1 Transaction Chains

A transaction chain is a sequence such as A→B→C→D. It shows that funds or transaction relationships move through several accounts. The chain may be relevant when transfers occur close together in time or amounts show a pattern that requires explanation. DFS can explore deep paths, while BFS can identify entities within a bounded number of hops.

7.2 Circular Patterns

A circular pattern is represented by a directed cycle, such as A→B→C→A. Cycle detection can locate the structure. Additional analysis should establish whether the timestamps, amounts, and transaction purposes support a meaningful interpretation. A cycle alone is not evidence of wrongdoing.

7.3 Star-Shaped Networks

In a star-shaped structure, one central account is connected to many other accounts. A high-degree account may receive funds from many sources or distribute funds to many destinations. Such a structure can be important, but it may also represent a legitimate merchant, payroll account, or payment service.

7.4 Dense Clusters

A dense cluster contains many internal connections relative to the number of possible connections. Community-detection methods may identify such groups. The analyst should determine whether the density is expected for the entities involved and whether transaction timing or other attributes add meaningful context.

7.5 Sudden Structural Changes

An account that suddenly develops many new connections, begins sending funds to unfamiliar destinations, or changes its transaction pattern may warrant review. Comparing graph snapshots over time can reveal such changes. Baselines should account for legitimate seasonal, business, or life-event changes.

7.6 Combining Indicators

A stronger investigative signal may arise when several independent indicators appear together—for example, a rapid transaction chain that also contains a cycle and unusual account behaviour. Even then, the result is a reason to investigate rather than a conclusive determination. Combining signals can reduce reliance on any single graph feature.

8. ILLUSTRATIVE CASE STUDY AND TRANSACTION NETWORK

8.1 Purpose and Assumptions

This example uses a hypothetical network of six accounts, A through F. All amounts, times, and account labels are invented for teaching purposes. The example demonstrates how graph algorithms reveal structure; it does not claim that the network represents an actual fraud case.

8.2 Transaction Dataset

ID	From	To	Amount	Time
T1	A	B	₹9,00,000	09:00
T2	B	C	₹8,80,000	09:12
T3	C	D	₹8,70,000	09:25
T4	D	B	₹1,00,000	10:10
T5	A	E	₹25,000	10:20
T6	E	F	₹24,000	10:35
T7	F	C	₹20,000	10:50
T8	B	E	₹15,000	11:05

Table 4: Hypothetical transaction dataset.

8.3 Graph Representation

The vertex set is V = {A, B, C, D, E, F}. Each transaction creates a directed edge. The graph contains eight transaction edges. If every transaction is preserved, the edge list retains the individual amount and timestamp for each edge.

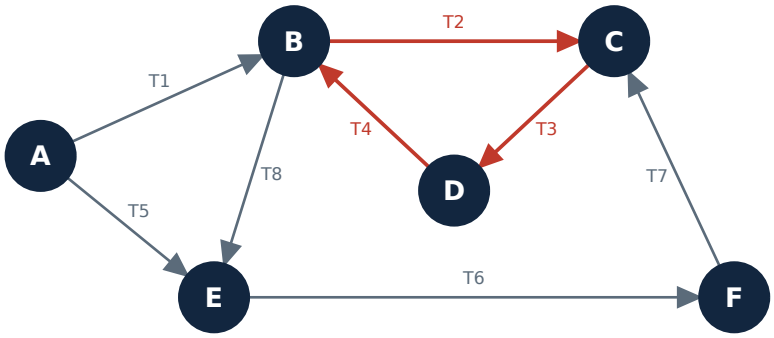


Figure 1: The hypothetical network. Edges are labelled with transaction IDs; the directed cycle B→C→D→B is highlighted in red.

8.4 Adjacency List

- **A:** B, E
- **B:** C, E
- **C:** D
- **D:** B
- **E:** F
- **F:** C

The adjacency list shows the outgoing neighbours. Transaction amounts and timestamps remain attached to the corresponding edges in the full representation.

8.5 BFS Analysis

Starting at A, and processing neighbours in the listed order, BFS first visits A. It then discovers B and E. From B it discovers C (E has already been discovered); from E it discovers F; from C it discovers D. One possible order is A, B, E, C, F, D. This order depends on neighbour ordering.

The distance from A in number of edges is: A = 0; B and E = 1; C and F = 2; D = 3. BFS therefore shows which accounts can be reached within a selected number of transaction steps.

8.6 DFS Analysis

A DFS from A may follow A→B→C→D before backtracking, then explore E→F. The exact order depends on adjacency ordering and visited-state handling. DFS reveals deeper paths and can support the cycle-detection process.

8.7 Cycle Analysis

The graph contains the directed cycle B→C→D→B. A DFS-based cycle detector can identify this because the edge D→B reaches a vertex on the active DFS path. This is a structural observation only. The transaction timestamps show that B→C and C→D occur before D→B, but further context would still be required to interpret the pattern.

8.8 Degree Analysis

Out-degrees in this graph are: A = 2, B = 2, C = 1, D = 1, E = 1, F = 1. In-degrees are: A = 0, B = 2, C = 2, D = 1, E = 2, F = 1. B, C, and E have more incoming links than the remaining accounts. This may guide investigation, but degree alone cannot establish risk.

8.9 Initial Interpretation

The graph offers several review leads: a chain from A to D, a directed cycle among B, C, and D, and multiple links involving B, C, and E. An analyst would next examine transaction purposes, customer histories, normal business patterns, and data quality before drawing conclusions.

8.10 Centrality and Community Review

The cycle B→C→D→B forms a strongly connected group because each of these vertices can reach the others by following directed edges. E and F connect to the wider network through E→F→C and the additional links from A and B. The graph is small, so a human can inspect it directly; a larger graph would require systematic computation.

Degree centrality highlights accounts with multiple incoming or outgoing relationships. Betweenness centrality could help identify accounts lying on many shortest paths, while community detection could examine whether a larger network contains groups with unusually dense internal interactions. These measures should be calculated using a defined method and interpreted in context.

8.11 Illustrative Risk-Prioritization Logic

For demonstration, suppose an analyst prioritizes cases that meet multiple review conditions: a rapid multi-account chain, a detected cycle, and an account whose recent connections differ from its historical pattern. This is a qualitative rule for the example, not a validated scoring model.

- **First review:** inspect the B–C–D cycle and verify the transaction purpose and timing.
- **Second review:** trace A→B→C→D and confirm whether the amounts and account relationships are expected.

- **Third review:** examine E and F in context, including the links from A and B and the transfer to C.

8.12 Findings from the Example

The example demonstrates how graph representation makes relationships visible, BFS measures transaction hops, DFS explores paths, and cycle detection identifies circular structure. Degree and connectivity measures provide additional descriptions of the network. The example does not establish that any account is fraudulent; it shows how DSA can organize evidence for investigation.

9. ALGORITHMIC AND COMPLEXITY ANALYSIS

9.1 Notation

Let V denote the number of vertices and E the number of edges. Let n denote the number of records when a separate list of transactions is processed. Complexity describes how computational work or memory grows with input size; actual performance also depends on implementation, hardware, data distribution, and graph structure.

9.2 Complexity Summary

Algorithm / operation	Typical time complexity	Auxiliary space
Adjacency matrix storage	$O(V^2)$	$O(V^2)$
Adjacency list storage	$O(V + E)$	$O(V + E)$
BFS using adjacency list	$O(V + E)$	$O(V)$
DFS using adjacency list	$O(V + E)$	$O(V)$
Directed cycle detection via DFS	$O(V + E)$	$O(V)$
Connected components	$O(V + E)$	$O(V)$
Kosaraju / Tarjan SCC	$O(V + E)$	$O(V)$
BFS shortest path	$O(V + E)$	$O(V)$
Dijkstra with binary heap	$O((V + E) \log V)$	$O(V)$ auxiliary
Bellman–Ford	$O(VE)$	$O(V)$ auxiliary
Comparison sorting	$O(n \log n)$	Depends on algorithm

Table 5: Complexity summary.

9.3 Sparse and Dense Graphs

A sparse graph has relatively few edges compared with the number of possible edges; a dense graph has many. A financial network can have many accounts while each account interacts with a small subset of all accounts, making adjacency lists attractive. If a graph is dense, the memory trade-off may be different.

9.4 Practical Scalability

Large networks may require time windows, subgraph extraction, incremental updates, indexing, and distributed processing. Expensive centrality measures may be approximated or calculated only on relevant subgraphs. These choices should be documented because they can affect results.

10. COMPARATIVE ANALYSIS

10.1 Transaction-Level versus Graph-Based Analysis

Aspect	Transaction-level analysis	Graph-based analysis
Main unit	Individual transaction	Entities and relationships
Strength	Simple rules and features	Reveals connected patterns
Typical methods	Thresholds, rules, transaction attributes	BFS, DFS, cycles, centrality, communities
Limitation	May miss coordinated patterns	More complex and computationally demanding
Best use	Known transaction-level indicators	Network context and relationship analysis

Table 6: Transaction-level versus graph-based analysis.

These approaches are complementary rather than mutually exclusive. A practical system can apply transaction-level rules first, use graph analysis to add context, and combine both outputs for review.

10.2 BFS versus DFS

BFS explores level by level and is suitable for minimum edge-count distances in unweighted graphs. DFS explores deeply and is useful for path exploration, cycle detection, and several connectivity algorithms. Both take $O(V + E)$ time with adjacency lists for a complete traversal.

10.3 Adjacency Matrix versus Adjacency List

An adjacency matrix supports constant-time edge checks but consumes $O(V^2)$ memory. An adjacency list uses $O(V + E)$ memory and explores neighbours efficiently. For sparse transaction networks, an adjacency list is usually the practical default.

10.4 Choosing a Shortest-Path Algorithm

Use BFS when all edges have equal cost and the objective is the fewest edges. Use Dijkstra when edge costs are non-negative and have a meaningful interpretation. Use Bellman–Ford when negative weights are required. Monetary amounts should not be used as distance weights unless minimizing total amount is truly the intended question.

10.5 Strengths and Trade-offs

Graph analysis adds relational context, but requires good entity resolution, clear definitions of edges, and careful interpretation. Rule-based analysis is easier to explain in simple cases, while graph analysis can reveal patterns that span several accounts. A combined design often provides a more complete basis for investigation.

11. LIMITATIONS, RISKS, AND CHALLENGES

11.1 Data Quality and Entity Resolution

Incorrect, incomplete, duplicated, or delayed records can distort the graph. If two distinct people are mistakenly treated as one entity, unrelated transactions may appear connected. Conversely, inconsistent identifiers can split one entity into several vertices and hide real relationships. Data lineage and validation are therefore essential.

11.2 False Positives and False Negatives

Legitimate businesses can produce high-degree accounts, dense communities, and circular payments. A graph rule that flags every such structure will generate false positives. Conversely, a sophisticated or poorly observed pattern may not trigger any chosen rule. Evaluation should measure both types of error.

11.3 Scale and Timeliness

Large graphs may exceed the memory or processing limits of a single machine. Real-time systems also face a trade-off between rapid alerts and waiting for more context. Time-windowing, indexing, incremental graph updates, and distributed computation can help, but introduce design complexity.

11.4 Privacy and Security

Financial graphs can reveal sensitive relationships. Access should be restricted, identifiers protected, and retention limited to legitimate purposes. The system should log access and provide controls against misuse.

11.5 Interpretability and Human Oversight

A score or centrality value is not self-explanatory. Alerts should preserve the path, transactions, timestamps, and rules that produced them. Human review remains important, especially before consequential action is taken.

11.6 Algorithmic Limits

Traversal algorithms find structural relationships but do not determine intent. Centrality and community results depend on graph construction and algorithm choice. Time and amount attributes may need separate temporal or weighted analysis. The limitations should be stated clearly in any report or deployment.

12. ADVANCED EXTENSIONS AND FUTURE SCOPE

12.1 Real-Time and Temporal Graphs

Future systems can update graphs as transactions arrive and evaluate recent activity against historical patterns. Temporal graphs preserve when relationships occurred, allowing analysis of sequence, duration, and changes over time.

12.2 Graph Databases

Graph databases store entities and relationships in a form that supports relationship-oriented queries. They may simplify exploration of multi-hop paths and linked entities, though the choice of database should depend on workload, scale, governance, and operational requirements.

12.3 Distributed Graph Processing

Large transaction networks may require distributed storage and processing across multiple machines. Partitioning a graph is challenging because edges can connect vertices on different partitions. Distributed methods can improve scale but may introduce communication costs and more complex consistency requirements.

12.4 Machine Learning and Graph Neural Networks

Graph-based machine learning can combine structural features with transaction attributes. Graph Neural Networks (GNNs) learn representations by aggregating information from connected entities. These methods may identify complex patterns but require reliable labelled data, careful validation, explainability, and controls against bias. They are extensions to—not replacements for—the DSA foundation in this report.

12.5 Link Prediction and Explainability

Link prediction estimates whether a relationship may exist or form in the future, while graph embeddings represent nodes or relationships as vectors for downstream analysis. Explainable systems should provide human-readable reasons for alerts, such as the relevant path, unusual connectivity, or time-based change.

12.6 Responsible Future Development

Future work should evaluate the system on representative data, measure precision and recall, test performance at scale, and audit false positives across relevant customer groups. Any advanced model should be compared with simpler baselines to establish whether the added complexity provides measurable value.

13. CONCLUSION

This case study examined how Data Structures and Algorithms can be applied to the analysis of financial transaction networks. Financial systems contain relationships among accounts, customers, merchants, devices, and transactions, making graph theory a natural way to represent the network. By treating entities as vertices and transactions as directed edges, the system can preserve both the relationships and the direction of money movement.

The report discussed graph representations and the roles of BFS, DFS, cycle detection, connected components, strongly connected components, and shortest-path algorithms. It also examined centrality, community detection, and supporting structures such as hash tables, queues, stacks, sets, heaps, and priority queues. Complexity analysis showed why representation and algorithm choice matter when networks become large.

The hypothetical case study demonstrated that graph algorithms can reveal transaction chains, cycles, and accounts with multiple relationships. These findings can help structure an investigation and prioritize cases for further review. At the same time, the example emphasized that suspicious structure is not proof of fraud. Legitimate activity can create similar patterns, and conclusions require transaction context, data-quality checks, and appropriate evidence.

Graph-based analysis is most useful as a complement to transaction-level rules and investigative procedures. Its effectiveness depends on reliable data, careful modelling, scalable implementation, privacy safeguards, and transparent interpretation. Future extensions may include temporal graphs, graph databases, distributed processing, and graph machine learning.

In conclusion, this topic demonstrates the practical importance of DSA beyond basic data storage and retrieval. Graph theory and its algorithms provide a structured way to analyse relationships in complex systems and can contribute to more informed, scalable, and relationship-aware financial fraud detection.

14. REFERENCES

The following sources provide foundational material on algorithms, graph theory, network analysis, and financial crime. The report's transaction dataset is hypothetical and was created solely for illustration.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
2. Sedgewick, R., and Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
3. Kleinberg, J., and Tardos, É. (2005). *Algorithm Design*. Pearson.
4. Newman, M. E. J. (2018). *Networks* (2nd ed.). Oxford University Press.
5. Easley, D., and Kleinberg, J. (2010). *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press.
6. Tarjan, R. E. (1972). Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2), 146–160.
7. Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269–271.
8. Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
9. Financial Action Task Force (FATF). Guidance and publications on money laundering and financial crime. Official website: https://www.fatf-gafi.org/
10. National Institute of Standards and Technology (NIST). Computer security and risk-management resources. Official website: https://www.nist.gov/

Before submission, confirm the citation style required by your instructor. If the department requires a specific format (such as IEEE or APA), adjust punctuation, italics, and access dates consistently. Use official or library-accessible copies when verifying publication details.